

re-using server-side database on client
(client would just download the graph DB instead of re-indexing the repository
if re-indexing would be more expensive than downloading it).

Issue 534843 contains
performance measurements when accessing Kuzu DB depending on number of
concurrent requests. Based on these measurements, we should:

- Store Kuzu databases on fast SSDs on the same node which will serve the graph DB requests (instead of using e.g. NFS mounted filesystem)
- Keep pool of opened DB connections for recently used repositories

Zoekt-only / graph-only nodes

We should also add a setting to our "Zoekt node" models to mark them as "zoekt only", "kuzu only", or "zoekt and kuzu". A "kuzu only" node will not be allocated new zoekt indexes and vice versa. This will give our operators the most flexibility to roll out changes while keeping as much infrastructure shared as possible. It also keeps deployment simple for self-managed as they can choose to use a single "zoekt and kuzu". This will be particularly useful during our early rollout as we move more quickly with the knowledge graph rollout without fear of taking down our GA Zoekt service. Additionally it may help with long term scalability if the different processes require different resources (e.g. memory or CPU). Furthermore it could simplify our monitoring as separate services would be easier to correlate resource usage (or incidents) with changes in a specific service.

Storage estimation

In issue 534843
are listed sizes of kuzu DBs depending on repository size. Although the final size of Kuzu DB may vary (depending on additional indexes we create and final parser we use), it's probable that for most of repositories DB size should not exceed 10MB. Statistics about repository sizes can be found in this comment.

Rough estimation is that we will need 1GB to index 100 repositories, so 1TB to index 100 000 repositories.

Graph DB management

Apart from serving graph DB queries, the API layer on graph node will also allow graph DB management - creation, update, deletion of graph DBs.

On repository creation/update, a graph node (gitlab-zoekt service) will accept a task to parse the repository. Parsing will be done on the graph node. Because Kuzu doesn't support multiple read-write connections to the same DB, a new DB for the repository will be created in a separate directory and then when import is finished we just replace directories. Running the parser and DB management (replacing DB with a new one, deletion of DB) will be controlled by

gitlab-zoekt service.

Authentication and authorization

Knowledge graph service will not implement any authorization checks (whether a user can access a repository), any permissions checks should be done on Rails side. Access to knowledge graph service should be allowed only from GitLab Rails. For authenticating requests on graph node we could use JSON Web tokens similarly to how we use them between other components.

Knowledge graph layer

Access to graph nodes will be allowed only from GitLab Rails. On Rails side there will be an abstraction layer which:

- As an input it accepts a Cypher query and project ID
- It returns query result in response. It takes care of all business logic related to sending/processing the request.

Scalability and high availability

Scaling of graph nodes will depend on:

- available disk space on each node (this will be the main factor, at least for the beginning)
- number of open database connections at the same time: each database connection reserves a certain memory space - this is configurable when opening the database connection and we can choose how much memory will be used. For example we can allocate 100MB memory for big graph databases and 10MB memory for small databases.

For a node with 64GB RAM, we could keep >500 concurrently opened connections to big databases or >5000 connections to small databases.

There are multiple ways how to implement high availability for knowledge graph service, but we should make sure that:

- Kuzu DBs should be stored directly on the same node as knowledge graph service because of latency
- Queries for the same repository should be served by the same node (even if there are multiple replicas of the repository) because knowledge graph service will keep open DB connections for recently used DBs

Because of similarities between knowledge graph and Zoekt searching, we will extend existing Zoekt infrastructure to serve also graph databases:

- on server side, Zoekt search models and services will be separate from graph models and services. But because we want to re-use Zoekt infrastructure, Zoekt Node model will be associated both with Zoekt models and also with knowledge

graph models. It might look like this:

```
mermaid
classDiagram
    namespace ZoektModels {
        class Node
        class Index
        class Repository
        class Task
        class EnabledNamespace
        class Replica
    }
    namespace KnowledgeGraphModels {
        class KnowledgeGraphEnabledRepository
        class KnowledgeGraphReplica
    }

    Node "1" --> "" Task : hasmany tasks
    Node "1" --> "" Index : hasmany indices

    EnabledNamespace "1" --> "" Replica : hasmany replicas

    Replica "1" --> "" Index : hasmany indices

    Index "1" --> "" Repository : hasmany repositories
    Repository "1" --> "" Task : hasmany tasks

    KnowledgeGraphEnabledRepository "1" --> "" KnowledgeGraphReplica : hasmany replicas
    KnowledgeGraphReplica "1" --> "" Task : hasmany tasks
    Node "1" --> "" KnowledgeGraphReplica : hasmany graph replicas
```

Because for graph database, we don't need to keep repositories from same namespace on the same node, replication strategy will be simpler compared to Zoekt search. Models schema may change depending on needs, but the main point is to illustrate separation of Zoekt and knowledge graph models, while still re-using Zoekt Node model for both services.

Creating or updating a repository

Parsing will be done on Zoekt nodes. When a graph database for a project repository should be updated (either because it was changed or because its graph doesn't exist yet):

- For each repository GitLab maintains: a primary node for the repository (e.g. first replica), a list of replicas where the repository is synced, a list of replicas where the repository should be synced and a list of nodes from where the repository should be removed
- Each graph node periodically checks GitLab Rails for a list of tasks which should be executed on the node

- The graph node fetches repository files and executes the parser on these files
- The graph node stores parser's output data in the graph database
- The graph node reports back to GtiLab Rails that the graph database was updated
- Rails updates replica's status in DB and schedules task for other replica nodes to copy parsed data from the primary node

mermaid

sequenceDiagram

box Rails

participant A as Internal API

participant W as Worker

participant DB as Rails DB

end

box Graph node1

participant G as Graph API

participant P as Parser

end

box Graph node2

participant G2 as Graph API

end

W->>DB: find primary node for repoX

DB-->>W: node1

W->>DB: create task for node1 to index repoX

G->>A: GET (periodic check for DBs to parse)

A-->>G: return task to index repoX

G->>P: Parse repository (call parser library/CLI)

P-->>G: Graph data

G->>G: create new DB repoX.new

G->>G: if repoX already exists, close connections to the repo

G->>G: replace repoX with repoX.new

G-->>A: OK

A->>DB: update status/metadata in DB for replica and node

G2->>A: GET (periodic check for DBs to sync)

A-->>G2: return list of tuples (repo, source node): [repoX:node1]

G2->>G: GET repoX

G-->>G2: send repoX

Adding a new graph node

When we need to add a new graph node for some reason (e.g. to better load-balance requests or because of storage space):

- A new graph node is created and it self-registers to GitLab
- GitLab maintains rebalancing of repository DBs by updating list replicas and primary node for each repository in GitLab Rails DB. For repositories which should use the new node as a primary node, steps for each repository will be:
 - GitLab adds the new graph node to the list of replicas where the repository should be synced

- When replication is finished, it sets the new node as primary node for the repository
- Optionally it adds old/previous node to the list of nodes from where the repository should be deleted

Removal of a graph node

When a graph node is removed or when the node stops periodically checking-in, then GitLab will consider this node as removed. GitLab will update database records accordingly:

- For repositories which have the node set as primary node, it sets the primary node to one of replicas
- For repositories which have the node set as a replica, it removes the node from replicas and adds another node to the list of replicas (which triggers synchronization)

Concurrent database connections limits

By default Kuzu DB doesn't expect hundreds of concurrently opened DB connections on the same host. Number of concurrently opened DB connections on one host is limited by following factors:

- Virtual memory allocation - by default Kuzu allocates 8TB of virtual memory for each DB connection (to assure that even really huge DBs could be used). Because Linux virtual memory limit per one process is 128TB we would be able to open only 15 DB connections before hitting virtual memory limit. Because we don't need to support such huge DBs we can limit virtual memory by compiling Kuzu DB with DEFAULTVMREGIONMAXSIZE set to 1GB. All performance tests were done with 1GB setting.
- Memory buffer pool size - parameter used when opening DB connection which sets memory buffer allocated for the DB connection. It shouldn't be too small to make sure that Kuzu can perform also complex queries. During testing 50MB pool size was used, but we can use $(\text{total memory}) / (\text{max number of connections})$.
- Performance testing
also showed that Kuzu performs well even for higher number of concurrent connections if DB is already opened. If DB connection is not pre-opened, then performance degrades with number of concurrent requests and is sensitive to disk speed (because we attempt to load many DBs at the same time). But timings are still decent for 10 concurrent requests. In future, this can be mitigated by either pre-opening DBs which we expect to be used (for example when user opens a project in IDE or opens Duo Chat) or by horizontal scaling of graph nodes.

Querying knowledge graph

Kuzu supports Cypher query language, which is a common language used for graph databases.

All requests to graph nodes should go through GitLab Rails which takes care of

authentication (of end users) and authorization (checking if a user can access a repository). Whoever can read project's repository should have access to the graph database for this repository.

- In GitLab Rails there will be a simple abstraction layer/interface which can be used to query knowledge graph for a repository. This layer will take care of permission checks, finding primary node for the repository and sending the request to the graph node and processing the response. As an input it will accept a cypher query and a project ID (on which we want to run the query). Note: instead of using primary node, we could also use replica (as long as we always pick same replica).
- Although queries could be served also by any replica, the primary node for the repository should be used because any follow-up queries would re-use already opened DB connection
- Kuzu databases on graph nodes will be always opened in read-only mode for security reason (we need write mode only when updating the DB which will be done separately from querying it)
- There will be a maximum timeout set for running a Cypher query, e.g. 15s to mitigate any long-running requests and exhausting maximum number of DB connections

mermaid

sequenceDiagram

box Rails

participant T as Duo Chat Tool

participant A as Knowledge graph layer

participant DB as Rails DB

end

box Graph node1

participant G as Graph API

end

T->>A: Cypher query on repoX

A->>DB: find primary node for repoX

DB-->>A: node1

A->>G: GET (Cypher query for repoX)

G->>G: open repoX DB if not yet opened

G->>G: run Cypher query

G-->>A: query result

A-->>T: query result

Graph database schema versioning

Graph database schema will evolve during time. Ideally the repository parser (which will return graph nodes and edges) should be also versioned and this version should be part of the parser output. For each graph DB and each graph node will track in Rails database what schema version was used for parsing the repository.

Communication protocol

Because we will re-use Zoekt infrastructure, we will also use same protocol used by Zoekt. Zoekt currently uses REST, but is going to switch to gRPC. gRPC communication should be available also between Zoekt nodes (which we will need for copying databases to replicas).

Known limitations

Because separate file-embedded databases are used, this approach is not suitable for running a query across high number of repositories (for searching all repositories in a big group structure).

Observability

Graph nodes will use existing monitoring, tracing and logging mechanisms to make sure that we can monitor these nodes and have enough data to investigate potential issues.

Similar metrics as for Zoekt indexer/service would be a good starting point. Most important metrics will be: disk usage, memory usage, number of open DB connections, number of requests, average bootstrap speed, parsing time, replication time.

Alternative Solutions

Use one graph database per root namespace

All repositories in a top-level namespace would be stored in single graph database. A downside is that then we would still need to handle authorization in graph database, specifically on graph node level which would be much more complex (more details about this complexity are in "One graph database" section below).

One graph database

An alternative approach is deploying a graph database which would be used for storing knowledge graphs for all repositories. We originally planned this approach but it has some major downsides:

- Security: if one graph database is used, then all repositories would have to be stored in one big graph (we didn't find an open source solution where use multi-tenancy on repository level). This means that then we would have to make sure that each query accesses only nodes/edges which belong to the selected repository. Because we plan to allow usage of LLM-generated Cypher queries (not only usage of predefined queries), we would have to always parse, rewrite and sanitize each Cypher query. Even with that this solution would be prone to query injections and there is a risk that a query could access other repositories. More details about complexity of this task can be found in this comment
- Scalability: for SaaS there can be millions of repositories, if we eventually want to keep a knowledge graph for each repository, the database would be really huge - it would contain billions of nodes and edges. For Apache AGE

(which was our primary choice) we noticed performance degradation on creation of new records with growing size of database. This issue contains more details about performance measurements done on Apache AGE.

SECTION: engineering/architecture/design-documents/linux_patch_automation/_index.md

{{< engineering/design-document-header >}}

This document is a work-in-progress and proposes architecture changes for the GitLab.com SaaS. The goal of this document is to define the processes followed for applying security patches to the major Linux fleets supporting GitLab.com.

This document is intended to extend the Linux OS Patching runbook, where existing processes and patching cadences are defined. Please refer to that document for scope definitions.

Current state

As described in the Services section of the previously mentioned runbook. Most of the systems currently in scope lack meaningful automation to help us keep these systems up to date with security patching. This means that most patching done is currently reactive in nature, and done manually. This is time consuming, and represents considerable toil for our SRE teams.

Scope

This proposal looks to target Kernel and OS package vulnerabilities found on the VM instances that directly support the services running on GitLab.com. The primary fleets being targeted are as follows:

Service	Owner	Exposure	Maintenance	Impact	Automation
-----	----	-----	:-----:	:-----:	
Runner Managers	scalability:practices	internal	low	no	
HAProxy	productionengineeringfoundations	external	low	no	
Gitaly	gitaly	internal	high	no	
Patroni	reliabilitydatabasereliability	internal	low	no	
PGBouncer	reliabilitydatabasereliability	internal	low	no	
Redis	scalabilitypractices	internal	low	no	
Console	none	internal	low	no	
Deploy	none	internal	medium	no	
Bastions	none	external	low	no	

See the table in runbooks for additional details

Proposal

This proposal looks to implement a process that does the following:

1. Notifies system owners when patching is nearing it's due date, based on the previously defined patching cadence in the Linux OS Patching runbook. It should include recent

vulnerability findings in these notifications.

1. Provides a common place to implement patch automation tooling. We want to avoid duplication of work, and maintenance overhead of the process by using a common set of tools across systems where possible.

1. Once patching has been performed, new vulnerability reports for the system will be provided that confirm whether or not the identified issues have been resolved.

1. Provides the results to security and compliance teams.

Notifications

The below proposal is planned to be a stop-gap solution to get working system patching notifications in place while a permanent solution is built within VulnMapper.

I propose a script be written to take on the responsibility of creating GitLab issues that are assigned to the appropriate service owner team.

A general workflow that this might follow looks like:

1. The script looks for service configuration that defines patching and reboot cadences.
 1. It could get this either from the service-catalog, or configuration local to the script.
1. It looks for previously closed (or opened) patching notification issues for each service.
1. It queries Wiz for vulnerabilities related to each service.
1. If there are Critical or High severity vulnerabilities found for a system, and the system is due for patching based on the closure time of the previous patching issue, a new issue is created with the vulnerability findings attached.
1. Once patching is complete (as designated by a workflow label), Wiz is queried again with updated findings being attached to the issue.
1. Security / Compliance teams are notified (or assigned) on the issue for final sign-off.

A flow chart of how this proposal would work:

mermaid

graph TD

```
Cron[Execution for service X] --> ScriptStart
ScriptStart[Check for open patching issues <sub>1</sub>] --> IssueOpen
ScriptStart --> NoIssueFound
NoIssueFound[No existing issue] --> |Issue closed in the past X weeks?| DoNothing
NoIssueFound --> |No recently closed issues found <sub>2</sub>| QueryWiz
QueryWiz[Query Wiz <sub>3</sub>] --> |No issues found| DoNothing
QueryWiz --> |Critical or High vulnerabilities found| CreateIssue
IssueOpen[Issue is open] --> |Is in verify| AssignSecurityReview
IssueOpen --> |Isn't in verify| DoNothing
AssignSecurityReview[Assign for review <sub>5</sub>] --> |review by security| Closed
DoNothing[Do Nothing]
CreateIssue[Create Issue <sub>4</sub>]
Closed
```

1. We'll look for all patching issues that contain the ~Service:: label

associated with the script execution run. If none are open, we record the time that the last patching issue was closed.

2. Recently closed is defined as the current date minus the patching cadence specific for the service. We do this primarily as a mechanism to ensure we are only creating a single issue within our agreed upon patching cadence, preventing issue spam and considerable SRE toil.
3. We query Wiz for all vulnerabilities found, and then group them by a common label. Currently this is the gitlabcomservice label pulled in from GCP.
4. When creating a new issue, we do the following:
 1. Add a summarization of the vulnerabilities found for the fleet, grouped by CVE.
 1. Set a due date
 1. Assign the issue to the team (or manager of said team).
5. When the issue is moved to workflow-infra::verify, we attach an updated list of vulnerabilities to the issue, and assign to the user/team responsible for the review.

Patch automation

Proposal

I'm proposing that we use Ansible playbooks stored in a central repository, where they can be executed with CI pipelines to automate patch application where applicable.

Initially, the CI pipelines would likely be run by hand in response to patching notification issues that are created based on each service's defined patching cadence. As confidence is built in the process, these pipelines could be set to run on a schedule, where no SRE intervention is required.

Example Playbooks:

- HAProxy
- Bastion hosts

Pros:

- We can easily reuse common tasks such as:
 - Alert silencing / unsilencing
 - Chef enable / disable
 - Apt package upgrades
- Concurrency and update ordering is easily controllable with variables.
- System discovery is straightforward with the GCP inventory plugins.
- By preserving logs from the CI pipelines, we will retain the full Ansible playbook output "for free" in a common location.

Cons:

- Requires a user with full sudo privileges to be installed across all systems.
 - This can be somewhat mitigated by limiting where login is allowed from to specific systems.
- This would likely only work with systems we manage with Chef.

Ansible Execution

Given the elevated privileges required to execute Ansible across the majority of our fleet, care must be taken regarding where execution is run from, and who can access the private keys allowing login. The following risk mitigation techniques will be used:

1. The private key will be stored in Vault with limited read access granted.
1. The key will only be used from a single project on the ops.gitlab.net GitLab instance and fetched from Vault.
1. Git commit and pipeline execution in this project will be restricted to infrastructure SREs that are responsible for patching operations.
1. The user provisioned for this purpose will have login restricted to a single network where runners are created for the Ops GitLab instance running in us-central1.
1. Protected environments will be used to require multiple approvals from qualified SREs before pipeline execution is allowed in the GPRD environment.
1. The repository will be subject to regular compliance auditing, by means of the completion of a Security Compliance Intake issue.

Alternatives considered

Ansible Pull

Pros:

- Because ansible-pull is pull based, no elevated credentials need to be installed across the fleet to coordinate patching.
- Common tasks could likely still be reused among nodes.

Cons:

- Because ansible-pull uses cron, it would be difficult (or impossible) to control when the patches would be applied on each node.
- This would likely only work with systems we manage with Chef.

Internally developed patching scripts

It was considered that we could develop scripts that perform the same actions as Ansible, but using scripts maintained ourselves.

Pros:

- We could provision users on all instances that have limited privileges to the systems.
 - A user could be installed with only sudo privileges to the commands required for patching the system.
- The scripts could still be called against remote instances from a CI pipeline.
- Concurrency and update ordering can still be controlled.
- We could script around systems that aren't Chef managed VMs.

Cons:

- Maintenance overhead is likely to be significant.
- The process is going to be more fragile compared to using an off-the-shelf tool like Ansible
- A user with login access is still going to need to be installed on each system. We're only limiting the access that the user has with this method.

SECTION: engineering/architecture/design-documents/logging/_index.md

{{< engineering/design-document-header >}}

This document describes the current state of Logging for GitLab SaaS Platforms, the challenges that we face as we continue to scale and grow, and a recommendation on the next steps.

Scalability Themes

- Observability across the Production Fleet is accessible for all
- Paved roads are the default for all team members
- Solutions at GitLab follow the Well Architected Services Framework

Overview

Observability is the ability to measure the state of a system by its outputs. Most Observability systems involve multiple different tools to attain that goal, with two of the most common systems being metrics and logs.

Metrics are a measurement of a service captured at runtime, usually aggregated to provide an overview of a system.

Logs are a recorded message of an event.

Each of these tools has benefits and downsides, and using them appropriately makes maintaining an Observability stack easier. As an example, logs can have infinite cardinality with almost no overhead in the system, whereas metrics systems are significantly impacted by high cardinality. Due to the fragility of our logging structure, short retention, and inadequate tooling standards, GitLab has frequently used metrics when logs would be a better solution. Extremely high cardinality metrics have and continue to be a significant problem for our metrics stack, and transitioning those to logs would improve the performance and usability of both systems.

We also store different data in different locations, and there's a great deal of confusion when metrics (which are almost always approximations) don't match up with logs (which are exact details of the requests). Having a standard of what goes where and keeping data in the system that is most appropriate would lower confusion and lower cost as we wouldn't be duplicating data in multiple places.

Every log and metric that we write adds to the complexity and cost of our observability systems. Using the right tool for the right job lessens the overall cost to GitLab and the complexity of management.

Current State of Logging

GitLab presently uses different logging solutions across its various platforms, both SaaS and Self-Managed as described below. The lack of standardization here makes it challenging to use and maintain. Without an opinionated recommendation on which logging tool to use, every team picks the one they are most familiar with or that is easiest for them to spin up.

- Security: Devo
- GitLab.com: Elastic Cloud, BigQuery, Google Cloud Logs, GCS, Cloudflare, Sentry, Loki POC
- GitLab Observability: ClickHouse)
- GitLab Dedicated: CloudWatch, BigQuery, OpenSearch, Cloud Logging.

Security

We are researching how security ingest and consume the logs that the production platforms produce.

GitLab.com

Logs: User Segment and Size Analysis has a great deal of research into the current use cases for logging.

No standardization

We presently have different logging schemas depending on who is outputting the logs, which makes using the logs for debugging difficult. As an example, there is no standard for which field holds the status code of a request, and no standards on what each logging severity should mean. This encourages a significant expansion in the number of fields that we index, which increases the cost of the logging cluster.

Different types of logs go into different systems. We presently use Elastic Cloud mostly for low to medium volume logs that need to be analyzed, however, HAProxy access logs go into BigQuery and Cloudflare logs are only available using their APIs. Each system we use also complicates the story on granular access.

We also have additional requests for new systems to bring online, such as Runway.

Limited features

Our logging environment only keeps data for 7 days, which is not enough for many debugging and analysis needs.

We do not presently have a good story for managing granular access, which means that bringing on additional types of logs can be impossible.

Limited observability

We do not have an easy way to track who is writing logs to the system and at what volume. We can get this data if we need to but it takes a great deal of time and effort.

We do not know who is using the system regularly and for what purposes. Being able to tell who

is using the data allows us to make better decisions on how to manage the cluster. Unfortunately for our team's name, we lack measurements of the system's availability.

Challenging management and growth

Management of only the Elastic Cloud logging system uses multiple different tools (terraform, Jsonnet + CI, shell scripts, Elastic Cloud UI).

Cells needs its own story that should be somewhat similar to Dedicated.

GitLab Dedicated

The Dedicated team would like to pass off some of the management of their logging clusters. This is complicated by their strict rules for granular access and some indices with RED data.

GitLab Observability

The Monitor:Observability team has a logging blueprint of their own.

Requirements for future

General requirements

One of our strongest requests has been for more retention. We believe 28 days would be ideal, however, some use cases such as error budget debugging may be improved by a retention rate of 30 - 45 days.

The system needs to be easy to manage and use Infrastructure as Code as a source of truth.

We should balance costs with the importance of making debugging problems easier.

The system should be flexible enough to adapt to compliance or regulatory requirements around log storage.

Paved Roads

We need to improve our tooling to make outputting the right logs easier.

- Standardized logging schema for GitLab logs
- Standardized system for outputting logs and metrics

We also need to make sure that there is an easy way to onboard new logging systems and a standardized set of tools to use. A well known single pane of glass to access logging will assist with this effort.

Granular Access

One of the significant issues with multiple different tools is that there isn't a single source of truth system for Granular Access. Using OKTA as our single source of truth matches with what GitLab already uses for internal tooling. This allows us to use systems that are already

in place for access requests, and limits the number of accounts we need to manage.

Auditing

One of the most significant gaps in our current logging implementations is auditing. We have the ability to know who is accessing the system over a span of time, but we do not know what they are using the system for. We also lack the ability to closely monitor log output volume and how often the logs are being used.

If we create an audit trail of user behavior and logging behavior, we create the possibility of chargebacks for the future.

We could also monitor for a significant change to log volume that would allow us to catch problems in logs before they become very expensive.

Observability

- Highly available and measured: Our team needs to set SLOs for all of our pieces
- System should work with our capacity planning and error budget tools to indicate when the system needs attention

Decisions we need to make

- Can we utilize and/or build similarly to the Monitor:Observability team?
- Centralized vs separated?
- Buy vs build?
- Which tools should we use?
- Can we design something that works for both GitLab.com and self-managed installations?

Next Steps

Create a unified Observability plan with Monitor:Observability team

The Monitor:Observability team has several Observability related blueprints: Observability overview, Logging, Metrics, and Tracing.

The use cases for their tooling and our tooling are different, and we recognize that GitLab.com is a more complex system and requires more robust tooling than the average self-managed customer. With that in mind, we should work closely with them to make certain we are Dogfooding anything possible.

Some efforts listed here (such as improving the structure of our logs) would likely benefit all customers and are something we should work on together.

Determine the correct tools to use

We should thoroughly investigate the tools that we are using today and the tools available on the market and narrow them down to a smaller number and standardize on only a few options. After that, we should create proofs of concept for each suggested tool and test them thoroughly.

Build vs buy?

This will be a question that we'll answer once we figure out the right tools.

Centralized vs decentralized?

A centralized access point is likely the correct solution, as it will allow us to have a single pane of glass to access all logging platforms. This will also allow us to have a single place to create audit logs of who is using the system and for what. It may make sense for Grafana to be that SPOG, but the tool we decide to use will influence that decision.

Create standards for logging and make it easy for applications to do the right thing

We need to design a structured logging system schema to make using our logs easier and create the appropriate tooling for applications within GitLab to output in the right format and to the right systems with as little effort as possible. If it's easy to do the right thing, people will do it by default. This will require organizational buy-in and training for the stage teams creating and maintaining features.

Simple Integration

It should be straightforward for service owners to use our logging solutions as a Managed Service.

SECTION: engineering/architecture/design-documents/machine_identity/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

GitLab currently supports more than 10 different bot user account types, each managed by different teams with inconsistent permission policies and authentication methods. This fragmentation creates challenges in ownership clarity, potential security vulnerabilities, customer visibility issues, and significant maintenance overhead. This proposal establishes service accounts as the core machine identity. Service accounts provide capabilities similar to regular users while enforcing important security constraints like UI sign-in restrictions and implementing streamlined permission policies. As part of this exercise, we would also need to ensure service accounts can be introduced on the free tier of GitLab with appropriate abuse prevention controls.

Motivation

A consolidated approach to machine identities through service accounts will:

- Simplify GitLab's machine access story by providing a unified interface and code path
- Enhance security through consistent authentication methods and access controls
- Reduce engineering overhead by centralizing bot-related code and policies
- Improve customer experience by allowing management of machine identities alongside other users
- Establish clear ownership and responsibility for machine identities
- Create proper isolation between human users and machine identities to facilitate easier ownership transfer and management.

Goals

- Enhance service accounts with features to support all required use-cases (e.g., project/group allow lists)
- Standardize on service accounts for all new machine identities
- Create migration paths for existing bot user types
- Simplify permission policies for machine identities
- Develop a revised monetization strategy for service accounts in collaboration with product teams
- Bring service accounts to GitLab Free tier with appropriate abuse prevention controls

Non-Goals

The following efforts are handled separately in their respective blueprints, and are out of scope for this proposal:

- Defining a Token Exchange Service
- Creating a low-privilege unified token as part of token consolidation

Machine identity consolidation, as proposed here, is already aligned with the direction of these initiatives.

Proposal

User type definition

To enforce clearer security and permission boundaries, standardize types into 2 categories:

- human users
- service accounts (machine identities): non-human accounts used for automation, integrations, and machine-to-machine interactions.

All internal bot-related functionalities will be consolidated under service account type.

This eliminates security inconsistencies by applying uniform authentication and access control across all external and internal machine identities.

Service accounts availability in the hierarchy

Service accounts are currently available at the top-group level on .com, and at

the instance level for self-managed.

To provide flexibility and support a wider range of use cases, service accounts should be available at all levels in the hierarchy:

- instance level
- organizations
- groups and subgroups
- projects

Instance level service accounts enable global automation and system-wide integrations. Group and project level service accounts allows granular control and access over machine identities.

Permissions

The current permission model of service accounts will continue to be applied across all hierarchy levels.

Owners of the namespace will be able to manage service accounts associated with that specific namespace, regardless of who created them.

Tiering

Service accounts are currently a Premium / Ultimate feature.

To fully support machine identities, Service accounts need to be available in the Free tier while preserving as much functionality as possible to prevent edge cases and code fragmentation.

However, additional limits may be necessary to prevent abuse and security risks in the Free tier. Token expiration for Personal access tokens:

- In Premium / Ultimate, token expiry for PATs of service accounts is not enforced. PATs can be created with optional expiration.
- In Free tier, expiration should always be required to prevent long-lived tokens from becoming a security risk. Keeping optional expiry as a Premium / Ultimate feature maintains a balance between flexibility and security controls.

Visibility

Service accounts are created at a specific point in the hierarchy and are only visible and accessible within that hierarchical branch.

Eg. a service account created at a subgroup-level will not be visible by sibling or parent groups, or unrelated organizations.

This restriction means that users outside of the service account's hierarchical branch won't be able to discover or add them as members, nor will the service account be able to access resources outside that scope.

This is considered a non-breaking change, as existing memberships remain unaffected.

However, with this change enabled, service accounts will no longer be accessible outside their hierarchical scope.

System-generated service accounts

System-generated service accounts will follow the same visibility and access rules as user-created ones.

Their management will also not differ: namespace owners will be able to manage them like any other service account.

The UI may indicate that a service account is system-generated or user-created for clarity.

Further restrictions

To prevent potential abuse, service accounts will have the following limitations:

- Service accounts do not have personal namespaces.
- Service accounts are prohibited from creating other service accounts.
This prevents unintended proliferation of machine identities, lateral movement from potential malicious actors, and maintains clear ownership.
- Service accounts are prohibited from creating top-level namespaces.
This ensures that all top-level groups remain under explicit human ownership.
- Free and trial namespaces will have a limit on the number of service accounts.
To support token consolidation (where tokens may be backed by a system-generated system account), tokens in these namespaces may also be subject to limits.

Open questions

- This proposal aligns with industry standards for service accounts, with a clear separation between human and machine identities, and leveraging the native access control mechanisms like memberships, roles, and inheritance.
It may still be valuable to further validate that we are following common patterns and meeting user expectations.
- How do we decouple membership and access? How do we support cross-project access? Is relying on multiple service accounts acceptable, or do we need to support cross-project access under a single token?

Alternative Solutions

See Alternative Solutions.

SECTION: engineering/architecture/design-documents/machine_identity/alternative_solutions.md

Alternative Solutions

Option 1 - using service accounts

In addition to the current proposal, we also considered the following additional features:

System-generated service accounts

To support internal features and automation, some service accounts will be created and managed automatically.

These system-generated service accounts differ from user-created service accounts in terms of control and visibility.

To prevent accidental disruption of application flows, system-generated service accounts will not be directly manageable by users or admins:

- they cannot be edited, deleted, or modified.
- their membership is automatically managed: the membership is added or removed based on the lifecycle of the feature or integration that depends on them.

This protects critical application flows from being broken due to misconfigurations or malicious actions (eg. removing a necessary system account from a project).

Creation and membership allowlists

Allowlists enable group or instance administrators to define and enforce boundaries on where service accounts can be created and used.

They can restrict:

- Creation: Define which groups or projects are permitted to create new service accounts.
- Membership: Define which groups or projects a service account may be added to as a member.

This mechanism helps ensure that service accounts are not unintentionally or maliciously created, or granted access to unrelated or sensitive parts of the hierarchy.

It also allows top-level owners to retain control over the footprint of service accounts created under their domain.

Note: these restrictions apply only to user-created service accounts. System-generated service accounts can be created regardless of the allowlist to avoid disrupting internal features and integrations.

Downsides

- Introduces 2 implicit types of service accounts (user-created vs system-generated).
This distinction could be made explicit (see Option 2).
- Access model issues persist: while allowlists mitigate them to an extent, they don't fully resolve the problem, and increase the risk of misconfiguration.
If a service account is added to multiple projects, neither the project

owners, nor the service account owner (top-level group owner) has visibility into where it's used or what access it has.

- Allowlists and the 2 implicit types adds significant complexity to both user experience and the codebase.

Option 2 - introduce a simplified user type

Service accounts have introduced significant complexity over time:

- It's a licensed feature
- They support multiple memberships, increasing the potential for accidental access sprawl.
- Ownership and membership are decoupled, with ownership limited to creation time and lacking visibility into access.
- User experience: administrative overhead

The current model requires a project owner to go through the following steps:

1. contact the instance admin and request a service account
2. instance admin creates a service account
3. project owner adds the service account as a project member
4. contact the instance admin again, and request a PAT for the service account
5. instance admin creates the PAT for the service account
6. instance admin has to securely transfer the token to the project owner as this cannot be done in gitlab-rails

Option 1 adds further complexity:

- It introduces a distinction between user-created and system-generated service accounts.
- It relies on allowlists to partially mitigate the downsides of multiple memberships and limited visibility into access.

Since service accounts are already generally available, we cannot change their behavior without disrupting existing user flows.

Instead:

- Preserve existing service accounts as-is for backward compatibility and as a machine identity option for users.
- Introduce a new user type (eg. service user) for all internal machine identity use cases, including token consolidation.

This service user would align more closely with the original proposal of service accounts:

- Available in the CE
- Single membership: a service user can only belong to a single group or project
- No distinction between ownership and membership: the namespace that creates the service user retains full control.

This approach results in a significantly cleaner access model, avoids the pitfalls of the existing service account implementation, and provides a lightweight path forward for automation needs.

Cross-group or cross-project operations would still be supported with this approach.

They would simply require a separate token for each group / project the service user needs to access.

If those groups belong to different organizations, cross-organization access using a single access token wouldn't be possible anyway, due to organization isolation.

Via introducing breaking changes over time, we could be simplify the existing service account model and eventually merge the two user types to reduce maintenance overhead.

Downsides

- Builds on 2 user types for machine identity, instead of consolidating under a single one.
- Despite the access model being simpler, it introduces some challenges. Eg. Amazon Q integration would require adding the service account to the top-level group, granting it access to all projects in the hierarchy.

Option 3 - some combination of 1 & 2

As an example: starting from a defined cutoff date, new memberships for service accounts cannot be created.

This preserves current access while simplifying future behavior without introducing a new user type.

SECTION: engineering/architecture/design-documents/mlops/_index.md

{{< engineering/design-document-header >}}

This blueprint describes GitLab end-to-end MLOps platform architecture, designed to support the complete machine learning lifecycle from experimentation to production deployment. This initiative supports our SaaS instance and self-managed instances while maintaining our "single application" philosophy.

Summary

GitLab MLOps is an integrated platform that provides end-to-end machine learning lifecycle management capabilities within GitLab's single application. It extends GitLab's existing CI/CD and registry capabilities to support ML workflows from experimentation to production and observability.

Motivation

Organizations face several key challenges when operationalizing ML:

1. Reproducibility: Data scientists struggle to track experiments and recreate results
1. Collaboration: Disconnect between data science, engineering and governance teams slows development
1. Deployment: Manual, error-prone processes for moving models to production
1. Governance: Difficulty maintaining oversight of model development, deployment and impact

These challenges often result in:

- Extended time-to-production for ML models
- Inconsistent development practices
- Security and compliance risks
- Resource inefficiencies

Goals

- Provide end-to-end ML lifecycle management integrated with existing GitLab DevOps workflows
- Provide a Model Registry - a place to store model versions, runs, metadata and artifacts
- Enable deployment of model versions from the model registry using CI/CD pipelines
- Enable importing models from Vertex and Huggingface to GitLab model registry
- Limited compatability with MLflow client for model experiments and registry

Non-Goals

- Providing extensive computation resources for model training beyond GPU runners
- Providing a model serving infrastructure
- Implementing feature stores
- Implementing data stores and becoming a dataops platform
- Developing a full-fledged MLflow server by achieving 100% MLflow API compatibility
- Model monitoring and tracing

Proposal

GitLab will provide a comprehensive MLOps platform built on top of existing GitLab infrastructure, leveraging and extending our CI/CD capabilities, package registry for artifact storage. The platform will support the full ML lifecycle through dedicated components while maintaining GitLab single application philosophy.

Design and Implementation Details

Component Architecture

```
mermaid
graph TB
    subgraph DevPhase["Development Phase"]
        direction TB
        A1[Experiment Tracking]:::ongoing
```

```

    A2[Model Registry]:::ongoing
    A3[GPU Runner Management]:::ongoing
    A4[Code repository]:::completed
end

subgraph CiCd["CI/CD Pipeline"]
    B4[Deployment Pipeline]:::new
end

A1 --> A2
DevPhase --> CiCd
A3 --> A1
A4 --> A1
CiCd --> DevPhase

%% Define styles for different statuses
classDef completed fill:a3cfbb,stroke:178344,color:black
classDef new fill:ffdebd,stroke:ff8c00,color:black
classDef ongoing fill:b8d0ff,stroke:0066cc,color:black

%% Place the legend at the bottom with right alignment
subgraph Legend[" "]
    direction LR
    L3[Ongoing]:::ongoing
    L2[New]:::new
    L1[Completed]:::completed
end

%% Position the legend at the bottom right
style Legend fill:none,stroke:none

```

Diagram Notes

- Code Repository: This is the Git repository either remote or locally.
- Experiment tracking: Code produces runs, artifacts, metrics etc. the metadata is stored centrally in Experiment Tracking
- Model Registry: Uses Package Registry to store artifacts
- Deployment pipeline: These are triggered either via Model Registry or via Git triggers.

Core Components

1. Experiment Tracking (existing feature)

The experiment management system will track ML training runs and their parameters:

- Experiment tracking with metadata storage
- Metric logging and visualization
- Storing artifacts

- Compatibility with MLflow client
- Access control and security policies for model experiments based on existing roles, custom roles and model registry read and write permissions. See Roles and permissions for model registry and experiments
- Data stored as in the code repository for smaller data sets using git or larger sets using git LFS.

2. Model Registry (existing feature)

Central repository for ML model management: Model registry docs.

- Model versioning and tagging (link to docs)
- Model metadata and lineage tracking
- Model approval workflows using GitLab labels for models and versions
- Integration with CI/CD pipelines to allow training and deployment
- Access control and security policies for model registry based on existing roles, custom roles and model registry read and write permissions.
- Compatibility with MLflow client
- Model cards with freeform markdown descriptions
- Governance instruments
- Users can store large data files in model registry too next to their model version artifacts for example.
- Integration with GCP Vertex AI model registry

3. Connection to GPU resources (existing feature)

Link to GPU runners docs.

- Maintain compatibility with GitLab runner
- Ensure ease of use with GPU runners

4. Model Deployment

Automated model deployment pipeline:

- Container-based deployment
- Multi-variate testing support
- Canary deployments
- Rollback capabilities
- Environment management
- Integration with GCP Vertex AI for deployment

5. API Clients

- Gitlab MLOps client for Python
- Limited MLflow client support: Logging of metrics, artifacts. Creation of models, versions and runs.
- Command-line (cURL) support with the existing API

Integration Points

1. GitLab CI/CD Integration

- Provide training, evaluation and validation CI/CD templates for ML workflows using GitLab (GPU) runners.
- Predefined variables for ML operations
- ML-specific CI/CD stages

2. Issue Tracking Integration

- Model development issues
- Approval workflows

3. GitLab Package registry

- Used for storage of model artifacts

Deployment Options

MLOps will support self-managed installation, including support for air-gapped environments and GitLab.com deployment and GitLab Dedicated.

Development Guidelines

No additional need beyond GDK. You might need MLflow client and GitLab MLOps Python Client

Out of scope

- Full MLflow client compatibility
- LLMOps
- AgentOps
- Model Governance, Security and Compliance
- Container Registry Integration

Conclusion

This technical blueprint provides a framework for implementing a comprehensive MLOps platform within GitLab. The proposed architecture leverages GitLab existing strengths while adding ML-specific capabilities that enable organizations to effectively manage their ML workflows at scale.

SECTION: engineering/architecture/design-documents/modular_monolith/_index.md

{{< engineering/design-document-header >}}

Summary

The main GitLab Rails project has been implemented as a large monolithic application, using

Ruby on Rails framework. It has over 2.2 million lines of Ruby code and hundreds of engineers contributing to it every day.

The application has been growing in complexity for more than a decade. The monolithic architecture has served us well during this time, making it possible to keep high development velocity and great engineering productivity.

Even though we strive for having an approachable open-core architecture we need to strengthen the boundaries between domains to retain velocity and increase development predictability.

As we grow as an engineering organization, we want to explore a slightly different, but related, architectural paradigm:
a modular monolith design,
while still using a monolithic architecture
with satellite services.

This should allow us to increase engineering efficiency, reduce the cognitive load, and eventually decouple internal components to the extent that will allow us to deploy and run them separately if needed.

Motivation

Working with a large and tightly coupled monolithic application is challenging:

Engineering:

- Onboarding engineers takes time. It takes a while before engineers feel productive due to the size of the context and the amount of coupling.
- We need to use CODEOWNERS file feature for several domains but these rules are complex.
- It is difficult for engineers to build a mental map of the application due to its size. Even apparently isolated changes can have far-reaching repercussions on other parts of the monolith.
- Attrition/retention of engineering talent. It is fatiguing and demoralizing for engineers to constantly deal with the obstacles to productivity.

Architecture:

- There is little structure inside the monolith. We have attempted to enforce the creation of some modules but have no company-wide strategy on what the functional parts of the monolith should be, and how code should be organized.
- There is no isolation between existing modules. Ruby does not provide out-of-the-box tools to effectively enforce boundaries. Everything lives under the same memory space.
- We rarely build abstractions that can boost our efficiency.
- Moving stable parts of the application into separate services is impossible due to high coupling.
- We are unable to deploy changes to specific domains separately and isolate

failures that are happening inside them.

Productivity:

- High median-time-to-production for complex changes.
- It can be overwhelming for the wider-community members to contribute.
- Reducing testing times requires diligent and persistent efforts.

Goals

- Increase the development velocity and predicability through separation of concerns.
- Improve code quality by reducing coupling and introducing useful abstractions.
- Build abstractions required to deploy and run GitLab components separately.

How do we get there?

While we do recognize that modularization is a significant technical endeavor, we believe that the main challenge is organizational, rather than technical. We not only need to design separation in a way that modules are decoupled in a pragmatic way which works well on GitLab.com but also on self-managed instances, but we need to align modularization with the way in which we want to work at GitLab.

There are many aspects and details required to make modularization of our monolith successful. We will work on the aspects listed below, refine them, and add more important details as we move forward towards the goal:

1. Deliver modularization proof-of-concepts that will deliver key insights.
1. Align modularization plans to the product structure by defining bounded contexts.
1. Separate domains into modules that will reflect product structure.
1. Start a training program for team members on how to work with decoupled domains (TODO)
1. Build tools that will make it easier to build decoupled domains through inversion of control (TODO)
1. Introduce hexagonal architecture within the monolith
1. Introduce clean architecture with one-way-dependencies and host application (TODO)
1. Build abstractions that will make it possible to run and deploy domains separately (TODO)

Status

In progress.

- A working group Bounded Contexts was concluded in April 2024 which defined a list of bounded contexts to be enforced for GitLab Rails domain and infrastructure layer.

Decisions

1. ADR-001: Modularize application domain? Start with modularizing the application domain and infrastructure code.

1. ADR-002: Define bounded context around feature categories as a SSoT in the code.
1. ADR-003: Assign stewards to all modules and libraries.

Glossary

- modules are Ruby modules and can be used to nest code hierarchically.
- namespaces are unique hierarchies of Ruby constants. For example, `Ci::` but also `Ci::JobArtifacts::` or `Ci::Pipeline::Chain::`.
- packages are Packwerk packages to group together related functionalities. These packages can be big or small depending on the design and architecture. Inside a package all constants (classes and modules) have the same namespace. For example:
 - In a package `ci`, all the classes would be nested under `Ci::` namespace. There can be also nested namespaces like `Ci::PipelineProcessing::`.
 - In a package `ci-pipelinecreation` all classes are nested under `Ci::PipelineCreation`, like `Ci::PipelineCreation::Chain::Command`.
 - In a package `ci` a class named `MergeRequests::UpdateHeadPipelineService` would not be allowed because it would not match the package's namespace.
 - This can be enforced easily with Packwerk's based RuboCop Cops.
- bounded context is a top-level Packwerk package that represents a macro aspect of the domain. For example: `Ci::`, `MergeRequests::`, `Packages::`, etc.
 - A bounded context is represented by a single Ruby module/namespace. For example, `Ci::` and not `Ci::JobArtifacts::`.
 - A bounded context can be made of 1 or multiple Packwerk packages. Nested packages would be recommended if the domain is quite complex and we want to enforce privacy among all the implementation details. For example: `Ci::PipelineProcessing::` and `Ci::PipelineCreation::` could be separate packages of the same bounded context and expose their public API while keeping implementation details private.
 - A new bounded context like `RemoteDevelopment::` can be represented a single package while large and complex bounded contexts like `Ci::` would need to be organized into smaller/nested packages.

References

List of references

SECTION: engineering/architecture/design-documents/modular_monolith/bounded_contexts.md

Historical context

Until May 2024 the GitLab codebase didn't have a clear domain structure. We have forced the creation of some modules as a first step but we didn't have a well defined strategy for doing it consistently.

The majority of the code was not properly namespaced and organized:

- Ruby namespaces used didn't always represent the SSoT. We had overlapping concepts spread across multiple namespaces. For example: `Abuse::` and `Spam::` or `Security::Orchestration::` and

Security::SecurityOrchestration.

- Domain code related to the same bounded context was scattered across multiple directories.
- Domain code was present in lib/ directory under namespaces that differed from the same domain under app/.
- Some namespaces were very shallow, containing a few classes while other namespaces were very deep and large.
- A lot of the old code was not namespaced, making it difficult to understand the context where it was used.

In May 2024 we defined and enforced bounded contexts.

Goal

1. Define a list of characteristics that bounded contexts should have. For example: must relate to at least 1 product category.
1. Have a list of top-level bounded contexts where all domain code is broken down into.
1. Engineers can clearly see the list of available bounded contexts and can make an easy decision where to add new classes and modules.
1. Define a process for adding a new bounded context to the application. This should occur quite infrequently and new bounded contexts need to adhere to the characteristics defined previously.
1. Enforce the list of bounded contexts so that no new top-level namespaces can be used aside from the authorized ones.

Iterations

1. Extract libraries out of the lib/ directory.
 - This step is non blocking to modularization but the less generic code exists in lib/ the easier will be to identify and separate bounded context.
 - Isolate code that could live in a separate project, to prevent it from depending on domain code.
1. ADR-001: Modularize application domain? Start with modularizing
1. ADR-002: Define bounded context around feature categories as a SSoT in the code.
1. ADR-003: Assign stewards to all modules and libraries.
1. Publish the list of bounded contexts.
 - Define a SSoT list of bounded contexts.
 - Enforce enforce it using RuboCop static analyzer.
 - Autoload non-standard Rails directories based on the given list.

SECTION: engineering/architecture/design-documents/modular_monolith/packages_extraction.md

The general steps of refactoring existing code to modularization could be:

1. Use the same namespace for all classes and modules related to the same bounded context.
 - Why? Without even a rough understanding of the domains at play in the codebase it is

difficult to draw a plan.

Having well namespaced code that everyone else can follow is also the pre-requisite for modularization.

- If a domain is already well namespaced and no similar or related namespaces exist, we can move directly to the next step.

1. Prepare Rails development for Packwerk packages. This is a once off step with maybe some improvements added over time.

- We will have the Rails autoloader to work with Packwerk's directory structure, as demonstrated in this PoC.

- We will have Danger-Packwerk running in CI for merge requests.

- We will possibly have Packer check running in Lefthook on pre-commit or pre-push.

1. Move file into a Packwerk package.

- This should consist in creating a Packwerk package and iteratively move files into the package.

- Constants are auto-loaded correctly whether they are in app/ or lib/ inside a Packwerk package.

- This is a phase where the domain code will be split between the package directory and the Rails directory structure.

We must move quickly here.

1. Enforce namespace boundaries by requiring packages declare their dependencies explicitly and only depend on other packages' public interface.

- Why? Up until now all constants would be public since we have not enforced privacy. By moving existing files

into packages without enforcing boundaries we can focus on wrapping a namespace in a package without being distracted

by Packwerk privacy violations. By enforcing privacy afterwards we gain an understanding of coupling between various constants and domains.

- This way we know what constants need to be made public (as they are used by other packages) and what can

remain private (taking the benefit of encapsulation). We will use Packwerk's recorded violations (like RuboCop TODOs) to refactor the code over time.

- We can update the dependency graph to see where it fits in the overall architecture.

1. Work off Packwerk's recorded violations to make r